

Kursi.ge

Home Assignment



Problem Statement: Correspondent Settlement Funding Selector

Kursi.ge's treasury desk funds outgoing correspondent-bank settlement instructions from a settlement account each morning. Each instruction consumes a certain amount of the account's available balance in the settlement currency and, if funded, earns the desk a processing fee.

The settlement account has a fixed remaining balance for the day (`availableSettlementBalance`). Your task is to build the backend service that selects the best combination of instructions, maximizing total expected fee without exceeding the account's available balance. Every funding request and its result must be persisted to a relational database so the treasury desk can audit past decisions.

Requirements

Inputs

A list of candidate instructions, where each instruction has:

- `instructionAmount` (in currency units): Settlement balance consumed if the instruction is funded.
- `expectedFee` (in currency units): Fee earned if the instruction is funded.
- `instructionReference` : Human-readable label for the instruction.

A maximum balance limit (`availableSettlementBalance`): the settlement account's total available balance for the day.

Objective

Find the combination of instructions that maximizes total expected fee while ensuring total instruction amount is less than or equal to `availableSettlementBalance` .

Output

- A list of instructions (`instructionReference` , `instructionAmount` , `expectedFee`) selected for the day.

- The total expected fee and total settlement balance consumed of the selected instructions.
- A unique request ID assigned to this funding run.

Example Input

```
{
  "availableSettlementBalance": 20000,
  "candidateInstructions": [
    { "instructionReference": "INS-2001", "instructionAmount": 7000, "expectedFee": 150 },
    { "instructionReference": "INS-2002", "instructionAmount": 9000, "expectedFee": 210 },
    { "instructionReference": "INS-2003", "instructionAmount": 4000, "expectedFee": 90 },
    { "instructionReference": "INS-2004", "instructionAmount": 6000, "expectedFee": 130 }
  ]
}
```

Example Output

```
{
  "requestId": "f1a2b3c4-d5e6-...",
  "selectedInstructions": [
    { "instructionReference": "INS-2001", "instructionAmount": 7000, "expectedFee": 150 },
    { "instructionReference": "INS-2002", "instructionAmount": 9000, "expectedFee": 210 },
    { "instructionReference": "INS-2003", "instructionAmount": 4000, "expectedFee": 90 }
  ],
  "totalSettlementConsumed": 20000,
  "totalExpectedFee": 450,
  "createdAt": "2026-09-07T09:00:00Z"
}
```

Constraints

1. If no combination fits within `availableSettlementBalance`, return an empty list and `totalExpectedFee = 0` with HTTP 200.
2. Instructions cannot be partially funded – you must take the full `instructionAmount` of an instruction if it's selected.
3. Return HTTP 400 with a descriptive error message for any invalid input.

Deliverables

A fully functional Spring Boot application built using Java or Kotlin. The application should:

- Have a REST API implementing all three endpoints listed below.
- Split logic into appropriate layers (Controller, Service, Repository, Algorithm).
- Use Flyway or Liquibase for versioned schema migrations.
- Include well-written unit tests and integration tests for logic and API endpoints.
- Be configured to run using Gradle or Maven.
- Include a Docker Compose file to set up the database.
- Run as a JAR file after being built. No Dockerization of the app itself required.

Upload the code to a public GitHub, GitLab, or Bitbucket repository. Include a README with:

- Clear instructions on how to build and run the application.
- Database setup.
- Example cURL requests and responses for all three endpoints.
- A brief explanation of the database schema design and index choices.

API Endpoints

1. `POST /api/v1/settlement/fund` – accepts the input payload above, runs the funding selection, persists the request and result, returns the output payload above with HTTP 201.
2. `GET /api/v1/settlement/{requestId}` – returns the persisted funding result for a given `requestId`, HTTP 200 if found, HTTP 404 if not.
3. `GET /api/v1/settlement` – returns a paginated list of all past funding runs (audit trail), newest first, HTTP 200.